

P1

```
int signMask(void) {  
    return 1 << 31;  
}
```

简单易得，略

P2

```
int bitXor(int x, int y) {  
    return ~(~(x & ~y) & ~(~x & y));  
}
```

取异或，先有思路 $x=0, y=1$ 或者 $x=1, y=0$ ，即 $(x \& \sim y) \mid \sim(x \& y)$ ，然后因为 $x \mid y$ 可转换为至少一个为 1，其反面为都为 0，即可得到 $\sim(x \& y)$ ，最后综合得到结果

P3

```
int negativePart(int x){  
    return (~x+1) & (x >> 31);  
}
```

易得 $x + \sim x + 1 = 0$ ，所以 $-x = \sim x + 1$ ，然后要求 $x > 0$ 取 0，否则取反，就可以取一个掩码 $x >> 31$ 刚好符合条件

P4

```
int copyByteWithin(int x, int src, int dst) {  
    int s = src << 3;  
    int d = dst << 3;  
    int t = (x >> s) & 0xFF;  
    return (t << d) | (x & ~(0xFF << d));  
}
```

整体思路就是提取 `src`，把 `src` 放到 `dst` 的位置，然后为了把 `src` 放进去，需要把原来的 `dst` 位置的 `byte` 清零

P5

```
int logicalShift(int x, int n) {  
    int mask = ~(((1 << 31) >> n) << 1);  
    return (x >> n) & mask;  
}
```

进行逻辑右移，先进行算术右移，高位再全部补 0，所以要设计一个掩码，让左数 n 位全部补 0

P6

```
int swapNibblePairs(int x) {  
    int mask = 0x0F | (0x0F << 8);  
    mask = mask | (mask << 16);  
    return ((x & mask) << 4) | ((x >> 4) & mask);  
}
```

```
}
```

构造掩码 0x0F0F0F0F, $x \& \text{mask}$ 可取低四位, $(x \gg 4) \& \text{mask}$ 取高四位

P7

```
int secondLowestZeroBit(int x) {  
    int z = ~x;  
    int rest = z & (z + ~0);  
    return rest & (~rest + 1);  
}
```

正难则反, 所以取反找第二个 1, 然后可以用 $z \& (z-1)$ 消去最低位置的 1, 然后继续找新的 rest 的最低的 1

P8

```
int oddParity(int x) {  
    x = x ^ (x >> 16);  
    x = x ^ (x >> 8);  
    x = x ^ (x >> 4);  
    x = x ^ (x >> 2);  
    x = x ^ (x >> 1);  
  
    return !(x & 1);  
}
```

因为 1^0 为 1, 1^1 为 0, 所以最后对半进行异或运算不改变 1 的个数的奇偶性, 即最后剩下 0, 代表偶数, 剩下 1, 代表奇数

P9

```
int rotateRightBits(int x, int n) {  
    int s = n & 31;  
    int s1 = (x >> s) & ~(((1 << 31) >> s) << 1);  
    int s2 = x << ((~s + 1) & 31);  
    return s1 | s2;  
}
```

思路较为清晰, 分成 2 部分各自计算再合并即可

P10

```
int roundEvenPow2(int x, int n) {  
    int half = 1 << (n + ~0);  
    int mask = (1 << n) + ~0;  
    int bias = (half + ~0) + ((x >> n) & 1);  
    return (x + bias) & ~mask;  
}
```

先将 x 分成 2^n 的整数倍部分和余数部分, 根据余数与半程 $2^{(n-1)}$ 的大小决定是否进位。如果刚好位于两个倍数中间, 则根据当前商的奇偶性决定是否进位

P11

```
int midpointTowardFirst(int x, int y) {
    int xy = x ^ y;
    int avg = (x & y) + (xy >> 1);
    int odd = xy & 1;
    int sx = x >> 31;
    int sy = y >> 31;
    int sd = sx ^ sy;
    int diff = x + ~y + 1;
    int ds = diff >> 31;
    int gtMask = ((~sx) & sy) | ((~sd) & (~ds));
    int gt = gtMask & !!diff;
    return avg + (odd & gt);
}
```

利用 $(x \& y) + ((x \wedge y) \gg 1)$ 计算不会溢出的向下取整中点。若 $x+y$ 为偶数则直接返回；若为奇数，则根据 x 与 y 的大小关系决定是否加 1。比较大小时需分别处理同号和异号情况，避免直接计算 $y-x$ 产生溢出。

P12

```
int isBetweenEitherOrder(int x, int a, int b) {
    int sx = x >> 31, sa = a >> 31, sb = b >> 31;
    int sxa = sx ^ sa, sxb = sx ^ sb;
    int dxa = x + ~a + 1, dax = a + ~x + 1;
    int dxb = x + ~b + 1, dbx = b + ~x + 1;
    int ltxa = (sxa & sx) | (~sxa & (dxa >> 31));
    int ltax = (sxa & sa) | (~sxa & (dax >> 31));
    int ltxb = (sxb & sx) | (~sxb & (dxb >> 31));
    int ltbx = (sxb & sb) | (~sxb & (dbx >> 31));
    return ((!ltxa) & (!ltbx)) | ((!ltxb) & (!ltax));
}
```

先分别判断 x 是否满足 $a \leq x \leq b$ 或 $b \leq x \leq a$ 。由于不能直接使用比较运算符，因此先比较符号位：若两数异号，则根据符号直接判断大小；若两数同号，则通过计算差值并观察其符号判断大小，从而避免减法溢出。最后将两种端点顺序的结果取或即可。

P13

```
int mul5Sat(int x) {
    int x2 = x << 1, x4 = x2 << 1;
    int y = x4 + x;
    int ov = ((x ^ x2) | (x2 ^ x4) | (x4 ^ y)) >> 31;
    int sign = x >> 31;
    int tmin = 1 << 31, tmax = ~tmin;
    int sat = (sign & tmin) | (~sign & tmax);
    return (ov & sat) | (~ov & y);
}
```

```
}
```

利用 $5x=(x<<2)+x$ ，不使用乘法。计算过程中分别得到 $2x$ 、 $4x$ 和最终的 $5x$ ，并通过比较这些结果与原数的符号位判断是否发生溢出；若发生正溢出则返回 INT_MAX ，负溢出则返回 INT_MIN ，否则返回正常的 $5x$ 。

P14

```
int classifyAdd3(int x, int y, int z) {  
    int s = x + y;  
    int sx = x >> 31, sy = y >> 31, ss = s >> 31;  
    int pos1 = ~sx & ~sy & ss;  
    int neg1 = sx & sy & ~ss;  
    int ov1 = (pos1 & 1) | neg1;  
    int t = s + z;  
    int sz = z >> 31, st = t >> 31;  
    int pos2 = ~ss & ~sz & st;  
    int neg2 = ss & sz & ~st;  
    int ov2 = (pos2 & 1) | neg2;  
    return ov1 + ov2;  
}
```

将 $x+y+z$ 拆成两次加法，先计算 $x+y$ ，再将结果与 z 相加。每次加法都通过操作数和结果的符号判断是否发生正溢出或负溢出，例如两个正数相加得到负数说明正溢出，两个负数相加得到非负数说明负溢出。最后综合两次加法的溢出方向，判断真实数学结果是超过 INT_MAX 、低于 INT_MIN ，还是仍在正常范围内。

P15

代码略，先从 uf 中提取符号位、阶码和尾数。若阶码全为 1，则输入为 NaN 或无穷，直接返回原值；对于非规格化数，直接对尾数进行 $3/2$ 倍运算；对于规格化数，则先补上隐藏的最高位 1，再进行 $3/2$ 倍运算。计算过程中按照 round-to-nearest-even 处理舍入，并根据结果调整阶码和尾数，最后重新组合得到浮点數位表示。

P16

先提取符号位、阶码和尾数，并根据阶码确定二进制小数点的位置。若原数已经没有小数部分则直接返回；否则根据指数确定需要舍弃的低位，将这些低位作为余数与半程值比较。余数小于一半时舍去，大于一半时进位，恰好等于一半时根据整数部分最低位的奇偶性决定是否进位，从而实现向偶数舍入，同时保留舍入为 0 时原有的符号。

P17

先判断 x 的正负并得到其绝对值，再寻找最高位 1 的位置，由此确定浮点数的实际指数和阶码。若整数有效位不超过 24 位，则通过左移直接构造尾数；若超过 24 位，则右移保留最高 24 位，并根据被舍弃部分进行

round-to-nearest-even 舍入。若舍入后有效位产生新的进位，则同时增加阶码，最后组合符号位、阶码和尾数。

P18

采用分组并行计数的方法统计二进制中 **1** 的数量。先利用掩码分别统计每 **2** 位中 **1** 的个数，再依次将相邻结果合并为每 **4** 位、**8** 位、**16** 位的计数，最终得到整个 **32** 位整数中 **1** 的总数，从而避免使用循环逐位统计。

P19

采用逐层交换的方法反转 **32** 位二进制序列。先利用掩码交换每组相邻的 **1** 位，再依次交换相邻的 **2** 位组、**4** 位组和 **8** 位组，最后交换高低 **16** 位。每一步都先用掩码提取对应部分，再通过左右移和按位或重新组合，最终完成所有位的完全反转。

实验截图

```
fanmo@LAPTOP-R7Q6LOMR:~/ICS/Lab1-DataLab$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 8/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 12/16 operators
roundEvenPow2: 14/24 operators
midpointTowardFirst: 23/32 operators
isBetweenEitherOrder: 44/48 operators
mul5Sat: 20/30 operators
classifyAdd3: 26/52 operators
floatScaleThreeHalves: 39/60 operators
floatRoundEven: 28/65 operators
float_i2f: 32/40 operators
bitCount: 28/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
fanmo@LAPTOP-R7Q6LOMR:~/ICS/Lab1-DataLab$
```

```
fanmo@LAPTOP-R7Q6LOMR:~/ICS/Lab1-DataLab$ ./btest
Score  Rating  Errors  Function
1      1      0      signMask
2      2      0      bitXor
3      3      0      negativePart
4      4      0      copyByteWithin
4      4      0      logicalShift
4      4      0      swapNibblePairs
4      4      0      secondLowestZeroBit
5      5      0      oddParity
5      5      0      rotateRightBits
5      5      0      roundEvenPow2
5      5      0      midpointTowardFirst
7      7      0      isBetweenEitherOrder
7      7      0      mul5Sat
7      7      0      classifyAdd3
7      7      0      floatScaleThreeHalves
10     10     0      floatRoundEven
10     10     0      float_i2f
10     10     0      bitCount
10     10     0      bitReverse
Total points: 110/110
fanmo@LAPTOP-R7Q6LOMR:~/ICS/Lab1-DataLab$
```